

Характеристики

Вызов инструмента и функции

 Copy page 

Используйте инструменты в своих подсказках

Вызовы инструментов (также известные как вызовы функций) дают большой языковой модели доступ к внешним инструментам. Большая языковая модель не вызывает инструменты напрямую. Вместо этого она предлагает инструмент для вызова. Затем пользователь вызывает инструмент отдельно и передает результаты обратно большой языковой модели. Наконец, большая языковая модель форматирует ответ в соответствии с исходным вопросом пользователя.

OpenRouter стандартизирует интерфейс вызова инструментов для всех моделей и провайдеров, что упрощает интеграцию внешних инструментов с любой поддерживаемой моделью.

Поддерживаемые модели: вы можете найти модели, поддерживающие вызов инструментов, отфильтровав их на openrouter.ai/models?supported_parameters=tools.

Если вы предпочитаете изучать полный сквозной пример, продолжайте читать.

Примеры тела запроса

Вызов инструмента с помощью OpenRouter состоит из трех основных этапов. Ниже приведены основные форматы тела запроса для каждого этапа:

Шаг 1. Запрос логического вывода с помощью инструментов

```
5     "role": "user",
6     "content": "What are the titles of some James Joyce books?"
7   }
8 }
```

... Посмотреть все 29 строк

Шаг 2. Выполнение инструмента (на стороне клиента)

После получения ответа от модели с помощью `tool_calls` выполните запрошенный инструмент локально и подготовьте результат.

```
1 // Model responds with tool_calls, you execute the tool locally
2 const toolResult = await searchGutenbergBooks(["James", "Joyce"]);
```

Шаг 3. Запрос логического вывода с использованием результатов инструмента

```
1 {
2   "model": "google/gemini-3-flash-preview",
3   "messages": [
4     {
5       "role": "user",
6       "content": "What are the titles of some James Joyce books?"
7     },
8   ],
9 }
```

... Посмотреть все 48 строк

Примечание: параметр `tools` должен присутствовать в каждом запросе (шаги 1 и 3), чтобы маршрутизатор мог проверять схему инструмента при каждом вызове.

Пример вызова инструмента

TypeScript SDK Питон Машинописный текст (выборка)

```
1 import { OpenRouter } from '@openrouter/sdk';
2
3 const OPENROUTER_API_KEY = "<OPENROUTER_API_KEY>";
4
5 // You can use any model that supports tool calling
6 const MODEL = "google/gemini-3-flash-preview";
7
8 ...
```

... Посмотреть все 23 строки

Определение инструмента

Далее мы определяем инструмент, который хотим вызвать. Помните, что инструмент будет *запрошен* большой языковой моделью, но код, который мы здесь пишем, в конечном счете отвечает за выполнение вызова и возврат результатов большой языковой модели.

TypeScript SDK Питон

```
1 async function searchGutenbergBooks(searchTerms: string[]): Promise<Book[]>
2   const searchQuery = searchTerms.join(' ');
3   const url = 'https://gutendex.com/books';
4   const response = await fetch(`${url}?search=${searchQuery}`);
5   const data = await response.json();
6
7   return data.results.map((book: any) => ({
8     ...
9     ...
10  }));
11 ...
```

... Посмотреть все 41 строку

Обратите внимание, что «инструмент» — это обычная функция. Затем мы пишем «спецификацию» в формате JSON, совместимую с параметром вызова функции OpenAI. Мы передадим эту спецификацию большой языковой модели, чтобы она знала о существовании этого инструмента и о том, как его использовать. При необходимости она

Использование инструмента и результаты его работы

Давайте выполним первый вызов API OpenRouter для модели:

TypeScript SDK Python TypeScript (fetch)

```
1  const result = await openRouter.chat.send({
2    model: 'google/gemini-3-flash-preview',
3    tools,
4    messages,
5    stream: false,
6  });
7
8  const response_1 = result.choices[0].message;
```

LLM отвечает причиной завершения `tool_calls`, а также массивом `tool_calls`. В общем обработчике ответов LLM вам нужно было бы проверять `finish_reason` перед обработкой вызовов инструментов, но здесь мы будем исходить из того, что так и есть. Продолжим и обработаем вызов инструмента:

```
4
5 // Now we process the requested tool calls, and use our book lookup tool
6 for (const toolCall of response_1.tool_calls) {
7     const toolName = toolCall.function.name;
8     const { search_params } = JSON.parse(toolCall.function.arguments);
9     const toolResponse = await TOOL_MAPPING[toolName](search_params);
10    messages.push({
11        role: 'tool',
12        toolCallId: toolCall.id,
13        name: toolName,
14        content: JSON.stringify(toolResponse),
15    });
16 }
```

Теперь в массиве messages есть:

1. Our original request
2. The LLM's response (containing a tool call request)
3. The result of the tool call (a json object returned from the Project Gutenberg API)

Теперь мы можем сделать второй вызов API OpenRouter и, будем надеяться, получим результат!

[TypeScript SDK](#) [Python](#) [TypeScript \(fetch\)](#)

```
1 const response_2 = await openRouter.chat.send({
2     model: 'google/gemini-3-flash-preview',
3     messages,
4     tools,
5     stream: false,
6 });
7
8 console.log(response_2.choices[0].message.content);
```

```
3 * *Ulysses*  
4 * *Dubliners*  
5 * *A Portrait of the Artist as a Young Man*  
6 * *Chamber Music*  
7 * *Exiles: A Play in Three Acts*  
8
```

Мы сделали это! Мы успешно использовали инструмент в подсказке.

Interleaved Thinking

Interleaved thinking allows models to reason between tool calls, enabling more sophisticated decision-making after receiving tool results. This feature helps models chain multiple tool calls with reasoning steps in between and make nuanced decisions based on intermediate results.

Important: Interleaved thinking increases token usage and response latency. Consider your budget and performance requirements when enabling this feature.

How Interleaved Thinking Works

With interleaved thinking, the model can:

- Reason about the results of a tool call before deciding what to do next
- Chain multiple tool calls with reasoning steps in between
- Make more nuanced decisions based on intermediate results
- Provide transparent reasoning for its tool selection process

Example: Multi-Step Research with Reasoning

Here's an example showing how a model might use interleaved thinking to research a topic across multiple sources:

```
3  "messages": [
4    {
5      "role": "user",
6      "content": "Research the environmental impact of electric vehicles and
7    }
8  ]
```

... See all 41 lines

Обоснование модели и вызовы инструментов:

1. Initial Thinking: "I need to research electric vehicle environmental impact. Let me start with academic papers to get peer-reviewed research."
2. First Tool Call: `search_academic_papers({"query": "electric vehicle lifecycle environmental impact", "field": "environmental science"})`
3. After First Tool Result: "The papers show mixed results on manufacturing impact. I need current statistics to complement this academic research."
4. Second Tool Call: `get_latest_statistics({"topic": "electric vehicle carbon footprint", "year": 2024})`
5. After Second Tool Result: "Now I have both academic research and current data. Let me search for manufacturing-specific studies to address the gaps I found."
6. Third Tool Call: `search_academic_papers({"query": "electric vehicle battery manufacturing environmental cost", "field": "materials science"})`
7. Final Analysis: Synthesizes all gathered information into a comprehensive response.

Best Practices for Interleaved Thinking

- Clear Tool Descriptions: Provide detailed descriptions so the model can reason about when to use each tool
- Structured Parameters: Use well-defined parameter schemas to help the model make precise tool calls
- Context Preservation: Maintain conversation context across multiple tool interactions

Implementation Considerations

When implementing interleaved thinking:

- Models may take longer to respond due to additional reasoning steps
- Token usage will be higher due to the reasoning process
- The quality of reasoning depends on the model's capabilities
- Some models may be better suited for this approach than others

A Simple Agentic Loop

In the example above, the calls are made explicitly and sequentially. To handle a wide variety of user inputs and tool calls, you can use an agentic loop.

Here's an example of a simple agentic loop (using the same `tools` and `initial messages` as above):

TypeScript SDK Python

```
1 async function callLLM(messages: Message[]): Promise<ChatResponse> {
2   const result = await openRouter.chat.send({
3     model: 'google/gemini-3-flash-preview',
4     tools,
5     messages,
6     stream: false,
7   });
8 }
```

... See all 47 lines

Best Practices and Advanced Patterns

Function Definition Guidelines

When defining tools for LLMs, follow these best practices:


```
2 { "name": "get_weather_forecast" }
```

```
1 // Avoid: Too vague
2 { "name": "weather" }
```

Подробные описания: предоставьте подробные описания, которые помогут модели понять, когда и как использовать инструмент.

```
1 {
2   "description": "Get current weather conditions and 5-day forecast for a sp
3   "parameters": {
4     "type": "object",
5     "properties": {
6       "location": {
7         "type": "string",
8         "description": "City name, zip code, or coordinates (lat,lng). Exam
9       },
10      "units": {
11        "type": "string",
12        "enum": ["celsius", "fahrenheit"],
13        "description": "Temperature unit preference",
14        "default": "celsius"
15      }
16    },
17    "required": ["location"]
18  }
19 }
```

Streaming with Tool Calls

When using streaming responses with tool calls, handle the different content types appropriately:

```
5     model: 'anthropic/claude-sonnet-4.5',  
6     messages: messages,  
7     tools: tools,  
8
```

... See all 40 lines

Tool Choice Configuration

Control tool usage with the `tool_choice` parameter:

```
1 // Let model decide (default)  
2 { "tool_choice": "auto" }
```

```
1 // Disable tool usage  
2 { "tool_choice": "none" }
```

```
1 // Force specific tool  
2 {  
3     "tool_choice": {  
4         "type": "function",  
5         "function": {"name": "search_database"}  
6     }  
7 }
```

Parallel Tool Calls

Control whether multiple tools can be called simultaneously with the `parallel_tool_calls` parameter (default is true for most models):

```
1 // Disable parallel tool calls - tools will be called sequentially  
2 { "parallel_tool_calls": false }
```

Multi-Tool Workflows

Design tools that work well together:

```
1 {
2   "tools": [
3     {
4       "type": "function",
5       "function": {
6         "name": "search_products",
7         "description": "Search for products in the catalog"
```

... See all 25 lines

Это позволяет модели естественным образом объединять операции в цепочки: поиск → получение подробной информации → проверка наличия товара.

Reliability Tracking

OpenRouter tracks how reliably each provider completes tool calls and surfaces this as the Tool Call Error Rate on the Performance tab of every model page. The same signal drives [Auto Exacto](#) provider ordering on tool-calling requests. For the exact validator, JSON Schema draft, regex semantics, and per-tool-call classification, see [How Tool-Calling Success Rate Is Measured](#).

For more details on OpenRouter's message format and tool parameters, see the [API Reference](#).